

MEMORIA DE PRÁCTICAS

Algoritmia y Complejidad · Curso 2025-2026

Las aventuras de Rubén por Alcalá

Optimización Logística Multi-Vehículo en Alcalá de Henares

Autores

Ruben Tostón Carrero · 09105509Q
Alejandro Cañas Sánchez · 09123538J
Miguel Huete Curvi · 03228161L

Escuela Politécnica Superior · Universidad de Alcalá
Mayo 2026

1. Introduccion

Este proyecto aborda un problema real de logística urbana mediante la combinación de cuatro algoritmos clásicos de algoritmia y complejidad, aplicados a un grafo dirigido y ponderado que representa la red viaria de Alcalá de Henares.

El sistema simula la jornada de Rubén, un repartidor de la empresa ficticia Haramball, que debe decidir qué paquetes cargar y en qué orden recorrer los barrios de la ciudad para maximizar el beneficio neto, descontando los costes operativos de cada vehículo disponible: mochila a pie, patinete eléctrico y furgoneta.

La solución es capaz de comparar automáticamente las tres opciones de transporte utilizando dos estrategias de selección independientes (Programación Dinámica y Greedy) y una búsqueda exacta de ruta (Backtracking con Branch & Bound), generando una recomendación final basada en la eficiencia económica (EUR/minuto).



2. Descripción del Problema

Haramball sigue el principio de "cuanto más caro sea el pedido, antes te llega a casa". Esto impone una política de prioridad explícita en la selección de pedidos y convierte el problema en una instancia del clásico problema de la mochila multidimensional combinado con el Problema del Viajante de Comercio (TSP).

2.1 Entidades del sistema

Entidad	Descripción
Almacén base	Nodo 0 – La Garena. Punto de partida y retorno de todas las rutas.
Nodo del grafo	12 ubicaciones de Alcalá de Henares interconectadas mediante aristas dirigidas y ponderadas (tiempo en minutos).
Pedido	Tupla (id, peso_kg, volumen_u, beneficio_EUR, destino_nodo). Cada pedido tiene restricciones físicas y económicas.
Vehículo	Define la capacidad máxima de peso y volumen, y el coste operativo por minuto (EUR/min).

Vehículo	Cap. Peso (kg)	Cap. Volumen (u)	Coste Operativo (€/min)
A pie	5	8	0.00
Patinete	15	20	0.3
Furgoneta	50	60	1.5

2.2 Mapa de nodos

El grafo representa 12 zonas de Alcalá de Henares interconectadas con distancias asimétricas (el tiempo de A→B puede diferir del de B→A, modelando sentidos únicos y condiciones de tráfico):

ID	Nombre	ID	Nombre	ID	Nombre	ID	
0	Garena (Almacén)	3	Campus	6	Ensanche	9	Nueva Alcalá
1	Cervantes	4	Magna	7	Chorrillo	10	Cuatro Caños
2	Estación	5	Espartales	8	Reyes Católicos	11	Hospital

3. Arquitectura del Sistema

El proyecto se estructura en cinco módulos independientes que se orquestan desde el script principal `simulacion_mejor_vehiculo()`. La separación de responsabilidades permite sustituir o mejorar cada componente de forma aislada.

Módulo	Responsabilidad
<code>floyd_warshall.py</code>	Pre-procesa el grafo y produce la matriz de caminos mínimos entre todos los pares de nodos. Ejecutado una sola vez por escenario.
<code>dp_seleccion.py</code>	Selecciona el subconjunto de pedidos óptimo respetando peso y volumen mediante mochila 0/1 bidimensional con Programación Dinámica.
<code>greedy_seleccion.py</code>	Alternativa voraz al módulo anterior. Ordena los pedidos por beneficio descendente (Quicksort) y los incluye si caben.
<code>backtracking_ruta.py</code>	Resuelve el TSP para los nodos de entrega seleccionados. Usa búsqueda exhaustiva con poda Branch & Bound para eliminar ramas inviables.
<code>multi_viaje.py</code>	Garantiza la entrega de TODOS los pedidos dividiendo la jornada en tandas sucesivas, reutilizando los módulos anteriores iterativamente.

3.1 Flujo de ejecución (pipeline)

Para cada par (vehículo × método de selección) el sistema ejecuta secuencialmente cinco pasos:

- PASO A — Selección de pedidos: DP o Greedy elige el subconjunto de mayor beneficio bruto respetando las capacidades del vehículo.
- PASO B — Coste de embalaje: se calcula como `FACTOR_EMBALAJE` (0,5 EUR/unidad de volumen) × volumen total cargado.
- PASO C — Ruta óptima (TSP): el módulo de `backtracking` recibe los nodos destino y minimiza el tiempo total del circuito. Dicho valor base se multiplica por el factor de velocidad del vehículo
- PASO D — Se calcula multiplicando el coste por minuto por el tiempo real de ruta calculado en el paso anterior (descontando combustible, desgaste, etc.).
- PASO E — Cálculo de eficiencia y expansión de ruta: `beneficio_neto / tiempo` y expansión de los tramos TSP con los caminos reales de Floyd-Warshall.

4. Entradas y Salidas del Sistema

4.1 Entradas

El sistema lee su configuración desde ficheros JSON ubicados en la carpeta escenarios/. Cada fichero define un escenario completo e independiente:

Campo JSON	Tipo	Descripción
nombre	string	Nombre descriptivo del escenario (p. ej. Lunes_normal).
descripcion	string	Breve texto explicativo del contexto del día de reparto.
matriz_adyacencia	list[list[float null]]	Matriz 12×12 de tiempos en minutos. null representa ausencia de conexión directa (se convierte a float("inf") al cargar).
pedidos[].id	int	Identificador único del pedido.
pedidos[].peso	int (kg)	Peso del paquete. Limita el vehículo que puede transportarlo.
pedidos[].volumen	int (u)	Volumen del paquete. Determina también el coste de embalaje.
pedidos[].beneficio	int (EUR)	Ingresos esperados al entregar el pedido. La política de Haramball prioriza los de mayor beneficio.
pedidos[].destino	int [0–11]	Índice del nodo de entrega en el grafo de Alcalá.

4.2 Salidas

Por cada par (vehículo, método) el sistema imprime y estructura en diccionarios Python los siguientes campos:

Campo de salida	Descripción
vehiculo / metodo	Identificadores del par analizado.
beneficio_bruto	Suma de beneficios de los pedidos seleccionados antes de descontar costes.
coste_embalaje	Coste proporcional al volumen total cargado (0,5 EUR/u).
coste_ruta	Coste operativo del vehículo: coste_por_minuto × tiempo_ruta.
beneficio (neto)	beneficio_bruto – coste_embalaje – coste_ruta.
tiempo	Duración total del circuito de reparto (minutos), devuelta por el TSP.
eficiencia	beneficio_netto / tiempo. Métrica principal de comparación entre vehículos.
ruta	Lista de nodos clave devuelta por el TSP (almacén → entregas → almacén).

ruta_expandida

Ruta con todos los nodos intermedios insertados por Floyd-Warshall (trayecto físico completo).

Ejemplo de salida con Furgoneta + DP:[DP] Pedidos: [1,3,5] | Neto: 142.5 EUR (bruto 185 - embalaje 15.0 - ruta 27.5) | Tiempo: 18.3 min | Eficiencia: 7.79 EUR/min

5. Algoritmos Implementados

5.1 Floyd-Warshall — Caminos mínimos entre todos los pares

Floyd-Warshall transforma la matriz de adyacencia original, que puede tener conexiones directas faltantes (inf), en una matriz de distancias mínimas reales. Se ejecuta una única vez por escenario, sirviendo de base a los demás módulos.

Complejidad temporal	Complejidad espacial
$O(n^3)$ con $n = 12$ nodos $\rightarrow$ 1.728 operaciones	$O(n^2)$ — dos matrices $n \times n$: distancias y predecesores

La función devuelve dos matrices: `dist[i][j]` con el coste mínimo y `pred[i][j]` con el predecesor inmediato de `j` en el camino desde `i`. La función `reconstruir_camino()` explota `pred` para materializar el camino, y `expandir_ruta_completa()` la aplica tramo a tramo sobre la ruta TSP para obtener el itinerario físico de Rubén.

```
# Núcleo del algoritmo ( $O(n^3)$ )
for k in range(n):          # nodo intermedio candidato
    for i in range(n):
        for j in range(n):
            if dist[i][k] == inf: continue # poda
            nuevo = dist[i][k] + dist[k][j]
            if nuevo < dist[i][j]:
                dist[i][j] = nuevo
                pred[i][j] = pred[k][j] # actualizar predecesor
```

Para dar más realismo a la simulación, y probar su robustez, el script [main.py](#) interviene antes de que el algoritmo de Floyd-Warshall sea usado. Aplica una calle cortada entre los nodos 4 y 5 (magna y Espartales), asignando valor infinito a la arista. Así, de forma dinámica, el algoritmo de Floyd-Warshall que tenía los cálculos ya hechos se ve obligado a recalcular las rutas con este inconveniente, demostrando que se adapta a cambios reales como tal.

5.2 Programación Dinámica — Mochila 0/1 Bidimensional

Resuelve el subproblema de selección de pedidos cuando existen dos restricciones físicas simultáneas: capacidad máxima de peso y capacidad máxima de volumen. La solución DP garantiza el óptimo global, a diferencia del Greedy.

Complejidad temporal	Complejidad espacial
$O(n \cdot W \cdot V)$ n = pedidos, W = cap. peso, V = cap. volumen	$O(n \cdot W \cdot V)$ — tabla 3D completa. Posible optimización a $O(W \cdot V)$ con tabla rolling.

La relación de recurrencia decide para cada pedido `i` si se incluye o no:

```
dp[i][w][v] = max(
    dp[i-1][w][v],          # no incluir pedido i
    beneficio_i + dp[i-1][w-peso_i][v-vol_i] # incluir pedido i (si cabe)
)
```

Tras rellenar la tabla, se reconstruye la solución recorriendo en sentido inverso desde `dp[n][W][V]` y comprobando si el valor cambió respecto a la fila anterior.

5.3 Selección Greedy con Quicksort

Alternativa de menor coste computacional al módulo DP. Ordena los pedidos por beneficio descendente (con Quicksort implementado manualmente) y los incluye en orden si caben simultáneamente en peso y volumen.

Complejidad temporal	Garantía de optimalidad
$O(n \log n)$ promedio $O(n^2)$ peor caso (Quicksort)	NO garantiza óptimo. Puede perder combinaciones de menor beneficio individual pero mayor beneficio conjunto.

El criterio de desempate en la ordenación es el "bulto" (peso + volumen) ascendente: entre dos pedidos de igual beneficio, se carga primero el más pequeño para maximizar el espacio restante y permitir incluir más artículos.

5.4 TSP con Backtracking y Branch & Bound

Resuelve de forma exacta el Problema del Viajante de Comercio (TSP) para los nodos de entrega seleccionados. El algoritmo explora recursivamente todas las permutaciones posibles de los nodos destino y aplica una poda agresiva: si el coste acumulado en cualquier rama supera el mejor resultado conocido, esa rama se abandona inmediatamente.

Complejidad teórica	Con poda
$O(n!)$ sin poda (n = nodos a visitar, no nodos del grafo)	La poda Branch & Bound reduce drásticamente el árbol de búsqueda. En la práctica, con 5-8 destinos, el tiempo es despreciable.

La función opera exclusivamente sobre la matriz de distancias mínimas entregada por Floyd-Warshall, garantizando que cada entrada $\text{dist}[i][j]$ ya es el coste real del camino más corto entre i y j , con independencia de si existe arista directa.

```
def explorar_rutas(lugar_actual, visitados, ruta_actual, coste_actual):
    # PODA: rama más cara que el récord → abortar
    if coste_actual >= mejor_coste_global: return
    # CASO BASE: todos los clientes visitados
    if len(visitados) == len(nodos_intermedios):
        coste_total = coste_actual + matriz[lugar_actual][origen]
        if coste_total < mejor_coste_global:
            actualizar_mejor(coste_total, ruta_actual + [origen])
        return
    # RECURSIÓN + BACKTRACKING
    for dest in nodos_intermedios:
        if dest not in visitados:
            visitados.add(dest); ruta_actual.append(dest)
            explorar_rutas(dest, visitados, ruta_actual,
                           coste_actual + matriz[lugar_actual][dest])
            ruta_actual.pop(); visitados.remove(dest) # deshacer
```

5.5 Multi-Viaje — Reparto Garantizado por Tandas

El módulo `multi_viaje.py` extiende el pipeline básico para garantizar que ningún pedido quede sin entregar, aunque la carga total supere la capacidad de un único viaje. En cada iteración selecciona los pedidos que caben (DP o Greedy), calcula la ruta TSP del circuito, y descarta esos pedidos de la lista de pendientes hasta que todos están entregados.

Si un pedido supera individualmente las capacidades del vehículo, se marca en `pedidos_sin_entregar` y no bloquea el resto del reparto.

6. Características Diferenciadoras

El sistema va más allá de implementar algoritmos aislados: los integra en un pipeline coherente que aborda un problema real con múltiples restricciones y criterios de optimización simultáneos.

Característica	Impacto
Dos restricciones físicas (peso + volumen)	La mayoría de implementaciones académicas de la mochila usan una sola dimensión. Aquí la mochila es genuinamente bidimensional, reflejando restricciones reales de un vehículo de reparto.
Costes operativos por vehículo	El beneficio bruto se penaliza con el coste de combustible/desgaste (EUR/min) y el coste de embalaje (EUR/u volumen), haciendo la comparación entre vehículos económicamente realista.
Acoplamiento Floyd + TSP	Floyd-Warshall corrige el grafo incompleto antes de que el TSP lo use, eliminando entradas inf que bloquearían rutas válidas a través de nodos intermedios.
Comparación DP vs Greedy	El sistema cuantifica empíricamente la brecha entre la solución óptima (DP) y la aproximada (Greedy), permitiendo al usuario evaluar cuándo merece la pena el mayor coste computacional de DP.
Escenarios JSON intercambiables	La separación entre datos y lógica permite probar cualquier configuración de ciudad y pedidos sin tocar el código, lo que facilita tests de regresión y benchmarking.
Garantía de entrega completa	El módulo multi_viaje.py asegura que todos los pedidos se entregan en tandas sucesivas, trascendiendo el modelo de un único viaje y acercándose a la realidad logística.

7. Formato del Fichero de Escenario JSON

El JSON de escenario reúne en un único fichero todos los datos necesarios para ejecutar una simulación completa. Su estructura está diseñada para que el código Python lo cargue directamente con la función `cargar_escenario()`, que convierte automáticamente los valores null en `float("inf")`.

El fichero `escenario_alcala_lunes.json` adjunto en el repositorio es el escenario de referencia construido a partir del grafo definido en el HTML interactivo del proyecto. Contiene 10 pedidos distribuidos por los 12 barrios del mapa.

7.1 Estructura del fichero

Los campos de primer nivel son:

- `nombre` (string): identificador legible del escenario.
- `descripcion` (string): contexto narrativo del día de reparto.
- `matriz_adyacencia` (array 12×12): tiempos entre nodos. null donde no hay conexión directa.
- `pedidos` (array de objetos): cada objeto define un paquete con `id`, `peso`, `volumen`, `beneficio` y `destino`.

7.2 Criterios de diseño de los pedidos

Los 10 pedidos del escenario de referencia han sido diseñados para ejercitar todos los casos de prueba relevantes:

- Pedidos de alto beneficio y bajo peso/volumen (ideales para cualquier vehículo).
- Pedidos de alto beneficio pero gran peso/volumen (excluyen la mochila o el patinete).
- Pedidos de bajo beneficio con alto volumen (candidatos a ser descartados por DP/Greedy).
- Destinos distribuidos por todos los barrios para forzar rutas largas en el TSP.
- Al menos un pedido demasiado pesado para la mochila, para validar los límites de capacidad.

El fichero JSON completo se puede consultar en el Anexo A. A continuación se muestra un extracto representativo de la estructura:

```
{
  "nombre": "Lunes_Alcala_Normal",
  "descripcion": "Día normal de reparto. 10 pedidos variados.",
  "matriz_adyacencia": [
    [0, 15, 10, 12, 8, 14, 11, 9, 6, 18, 16, 13],
    [45, 0, 8, null, 12, 18, 15, 10, 14, 7, 4, null],
    ... (12 filas en total)
  ],
  "pedidos": [
    {"id":1,"peso":2,"volumen":3,"beneficio":45,"destino":8},
    {"id":2,"peso":8,"volumen":10,"beneficio":70,"destino":3},
    ... (10 pedidos en total)
  ]
}
```

8. Análisis de Complejidad Global

La complejidad total por escenario combina la de los cuatro algoritmos involucrados. En la práctica, los parámetros son pequeños ($n \leq 12$, $W \leq 50$, $V \leq 60$, $\text{pedidos} \leq 20$), por lo que el sistema es completamente interactivo.

Módulo	Complejidad	Observación
Floyd-Warshall	$O(n^3)$	$n = 12$. Una sola ejecución por escenario.
DP Mochila 2D	$O(p \cdot W \cdot V)$	p pedidos, W peso máx, V volumen máx. Dominante en escenarios con muchos pedidos.
Greedy + Quicksort	$O(p \log p)$	Alternativa rápida. Peor caso $O(p^2)$ si Quicksort degenera.
TSP Backtracking	$O(d!)$ con poda	d = destinos seleccionados. La poda lo hace viable para $d \leq 10$.
Multi-Viaje	$O(T \times \text{alg})$	T = número de tandas. En la práctica $T \ll p$.

El cuello de botella teórico es el TSP (factorial), pero en la práctica la poda Branch & Bound lo controla eficazmente para el número de destinos manejado. El módulo DP puede volverse lento si se amplían mucho las capacidades de los vehículos, siendo Greedy la alternativa recomendada para escenarios de gran escala.

9. Conclusiones y Trabajo Futuro

9.1 Conclusiones

El proyecto demuestra cómo la combinación de algoritmos clásicos —Floyd-Warshall, Programación Dinámica, Greedy y Backtracking con Branch & Bound— puede abordar un problema logístico real de forma modular y extensible. La separación entre selección de pedidos y cálculo de ruta permite sustituir cualquier componente sin afectar al resto del sistema.

Los resultados obtenidos muestran que la Furgoneta es el vehículo más rentable cuando la demanda supera los 15 kg de carga, mientras que el Patinete domina en escenarios de pocas entregas ligeras. La métrica EUR/minuto es más informativa que el beneficio bruto, ya que penaliza los altos costes operativos de la Furgoneta en rutas cortas.

Anexo A — Escenario JSON de Referencia

El JSON siguiente es el fichero escenario_alcala_lunes.json, diseñado a partir de la matriz de adyacencia del grafo interactivo del proyecto. Cópialo en la carpeta escenarios/ de tu instalación para ejecutar la simulación.

El fichero completo con los 10 pedidos y la matriz 12×12 está disponible en el repositorio del proyecto como escenarios/escenario_alcala_lunes.json.

```
{
  "nombre": "Lunes_Alcala_Normal",
  "descripcion": "Lunes de reparto normal en Alcalá de Henares. 10 pedidos variados distribuidos por los 12 barrios del mapa. Combina paquetes ligeros de alto beneficio con bultos grandes que ponen a prueba las restricciones de peso y volumen de cada vehículo.",
  "matriz_adyacencia": [
    [ 0, 15, 10, 12, 8, 14, 11, 9, 6, 18, 16, 13],
    [ 45, 0, 8, null, 12, 18, 15, 10, 14, 7, 4, null],
    [ 10, 6, 0, 7, 5, 12, 9, 8, 7, 11, 8, 10],
    [ 12, null, 7, 0, 9, 8, 14, 16, 15, 22, 18, 3],
    [ 8, 10, 5, 9, 0, 6, 4, 8, 11, 15, 12, 11],
    [ 14, 16, 12, 8, 6, 0, 5, 11, 18, 20, 15, 9],
    [ 11, 13, 9, 14, 4, 5, 0, 6, 12, 16, 11, 15],
    [ 9, 12, 8, 16, 8, 11, 6, 0, 5, 14, 9, 18],
    [ 6, 15, 7, 15, 11, 18, 12, 5, 0, 9, 12, 17],
    [ 18, 9, 11, 22, 15, 20, 16, 14, 9, 0, 5, 24],
    [ 16, 5, 8, 18, 12, 15, 11, 9, 12, 4, 0, 60],
    [ 13, null, 10, 4, 11, 9, 15, 18, 17, 24, 5, 0]
  ],
  "pedidos": [
    {
      "id": 1,
      "peso": 2,
      "volumen": 3,
      "beneficio": 45,
      "destino": 8,
      "descripcion": "Smartphone — Reyes Católicos. Ligero y muy rentable. Cabe en todos los vehículos."
    },
    {
      "id": 2,
      "peso": 8,
      "volumen": 10,
      "beneficio": 70,
      "destino": 3,
      "descripcion": "Tablet + accesorios — Campus. Alto beneficio pero demasiado pesado para la mochila."
    },
    {
      "id": 3,
```

```
"peso": 1,  
"volumen": 2,  
"beneficio": 30,  
"destino": 10,  
"descripcion": "Libro técnico — Cuatro Caños. Muy ligero, beneficio moderado."  
},  
{  
  "id": 4,  
  "peso": 12,  
  "volumen": 18,  
  "beneficio": 95,  
  "destino": 9,  
  "descripcion": "Monitor 24" — Nueva Alcalá. Gran bulto y alto beneficio; requiere patinete o furgoneta."  
},  
{  
  "id": 5,  
  "peso": 3,  
  "volumen": 4,  
  "beneficio": 40,  
  "destino": 1,  
  "descripcion": "Auriculares inalámbricos — Cervantes. Cabe en cualquier vehículo."  
},  
{  
  "id": 6,  
  "peso": 20,  
  "volumen": 25,  
  "beneficio": 110,  
  "destino": 5,  
  "descripcion": "Impresora láser — Espartales. Solo cabe en la furgoneta. El más rentable por pedido."  
},  
{  
  "id": 7,  
  "peso": 5,  
  "volumen": 7,  
  "beneficio": 55,  
  "destino": 6,  
  "descripcion": "Teclado mecánico + ratón — Ensanche. Cabe en mochila (justo) o vehículos mayores."  
},  
{  
  "id": 8,  
  "peso": 6,  
  "volumen": 8,  
  "beneficio": 35,  
  "destino": 11,  
  "descripcion": "Set de altavoces — Hospital. Volumen medio, beneficio bajo; candidato a ser descartado."  
},
```

```
{
  "id": 9,
  "peso": 4,
  "volumen": 5,
  "beneficio": 60,
  "destino": 7,
  "descripcion": "Cámara fotográfica — Chorrillo. Excelente relación beneficio/peso."
},
{
  "id": 10,
  "peso": 15,
  "volumen": 20,
  "beneficio": 85,
  "destino": 4,
  "descripcion": "Televisor 32" — Magna. Solo patinete (justo) o furgoneta. Buen beneficio."
}
]
```